Data Provenance Canary: Project Proposal

Nov 3, 2026 · @sahasree

Summary

This project builds and evaluates a tool that lets a writer plant hidden "canaries" in their published text and later test whether a language model was trained on it. We fine-tune small open models on a corpus with canaries planted at controlled repetition counts, so we know the ground truth. We then measure which canary designs stay detectable after realistic training pipeline steps (deduplication, text normalization, paraphrasing) and under realistic access (full log probabilities vs. text output only). The central question: **how many copies of which kind of canary does a creator need to plant before training on their work becomes provable, and with what level of model access?**

Motivation and problem

Creators have no reliable way to show that a model was trained on their work. Model developers rarely disclose training data, and several copyright lawsuits against AI companies turn on exactly this question.

Existing membership inference attacks are a weak tool for this job. Recent evaluations show many of them perform close to random on large models once members and non-members come from the same distribution. Their apparent success often came from a date gap between the two sets rather than real memorization signal.

Canaries fix the ground truth problem: the creator generates them, so the creator knows exactly which text is unique to their work. What is not yet well understood is which canary designs survive a modern training pipeline, and whether they can be detected without access to log probabilities, which most commercial APIs do not provide.

Related work

Work	What it showed	Gap this project targets
Carlini et al., *The Secret Sharer* (2019)	Random canaries inserted in training data can be measured via "exposure"	Random strings are easy to filter out and don't look like real writing
Carlini et al., *Extracting Training Data from LLMs* (2021)	Models regurgitate rare training sequences verbatim	Extraction requires heavy repetition and generic prompts
Shi et al., *Min-K% Prob* (2024)	Low-probability tokens separate members from non-members	Needs log probabilities
Meeus et al., *Copyright Traps for LLMs* (2024)	Fluent synthetic "trap" sequences become detectable with enough repetitions	Not tested against dedup, normalization or text-only access
Wei et al., *Proving Membership via Data Watermarks* (2024)	Hypothesis tests on inserted watermarks give statistical proof	Watermark robustness to pipeline filters is open
Duan et al., *Do MIAs Work on LLMs?* (2024)	Many MIAs are near random once distribution shift is removed	Motivates canaries with clean ground truth
Maini et al., *LLM Dataset Inference* (2024)	Aggregating weak signals over many samples gives dataset-level verdicts	We apply the same idea to a single creator's canaries
Kandpal et al., *Deduplicating Training Data* (2022)	Deduplication sharply cuts memorization	Key threat model: dedup may erase repeated canaries

The novelty here is the combination: canary designs tested against realistic pipeline filters **and** realistic access levels, ending in a usable audit tool. The field moves fast, so week 1 includes a sweep of 2025 and 2026 work to confirm this gap is still open.

Research questions

1. **Dose:** How many repetitions does each canary type need before detection is reliable, and how does that change with model size?
2. **Survival:** Which canary types survive exact deduplication, near-duplicate filtering, Unicode normalization and partial paraphrasing?
3. **Access:** How much detection power is lost going from log probability access to text output only?
4. **Aggregation:** Can many individually weak canaries combine into a single dataset-level verdict with a controlled false positive rate?

Method

Canary types. Four designs, ordered from easiest to filter to hardest:

Type	Example	Detected by	Expected weakness
Random sequence	`kz7 qplm vortan 93x`	Loss, exposure	Obvious to humans and filters
Fluent trap sentence	LLM-generated sentence at a chosen perplexity	Loss, Min-K%	Removed by near-dup filters if repeated verbatim
Invisible watermark	Zero-width characters or homoglyphs in normal text	Token-level probability	Erased by Unicode normalization
Fictional fact	"The Varnholt Bridge opened in 1847."	Question answering	Weaker signal per canary, but survives paraphrase

The fictional fact canary is the main new design. It can be tested through plain question answering, so it works with text-only APIs.

Controlled training. Members and non-members are canaries drawn from the same generator, randomly split. This avoids the distribution shift problem that inflated earlier MIA results.

1. Build a base corpus of about 50 to 100 million tokens of public text (e.g. Wikipedia articles).
2. Insert each canary at 1, 4, 16 or 64 copies. Hold out a matched set of never-inserted canaries as non-members.

3. Apply pipeline variants: none, exact dedup, MinHash near-dedup, NFKC normalization, and 30% paraphrase of canary passages.
4. Continue pretraining Pythia-160M and Pythia-410M (and 1B if compute allows) on each variant.

Detectors. Two access levels:

- **Gray box (log probabilities):** loss, zlib ratio, Min-K%, Min-K%++, and loss calibrated against the untouched base model.
- **Black box (text only):** greedy prefix completion match, sampling-based perplexity estimates, and QA accuracy on fictional facts.

Dataset-level test. For each creator (a bundle of 50 to 200 canaries), compare member and non-member score distributions with a one-sided Mann-Whitney U test. Report a p-value as the audit verdict.

Experiments and evaluation

All canary types and repetition counts go into the same training run, so the grid is 5 pipeline variants × 2 model sizes = **10 training runs**.

Metric	What it answers
AUC per canary type and repetition count	RQ1: overall separability
TPR at 1% FPR	RQ1, RQ3: usefulness when false accusations are costly
Detection drop after each pipeline variant	RQ2: survival
Gray box vs. black box gap	RQ3: cost of losing log probabilities
Dataset-level p-value vs. number of canaries	RQ4: minimum canaries for $p < 0.01$
False positive rate on untouched base model	Sanity check: the test must not flag a model that never saw the canaries
Base corpus perplexity before and after	Canaries should not hurt the model's general quality

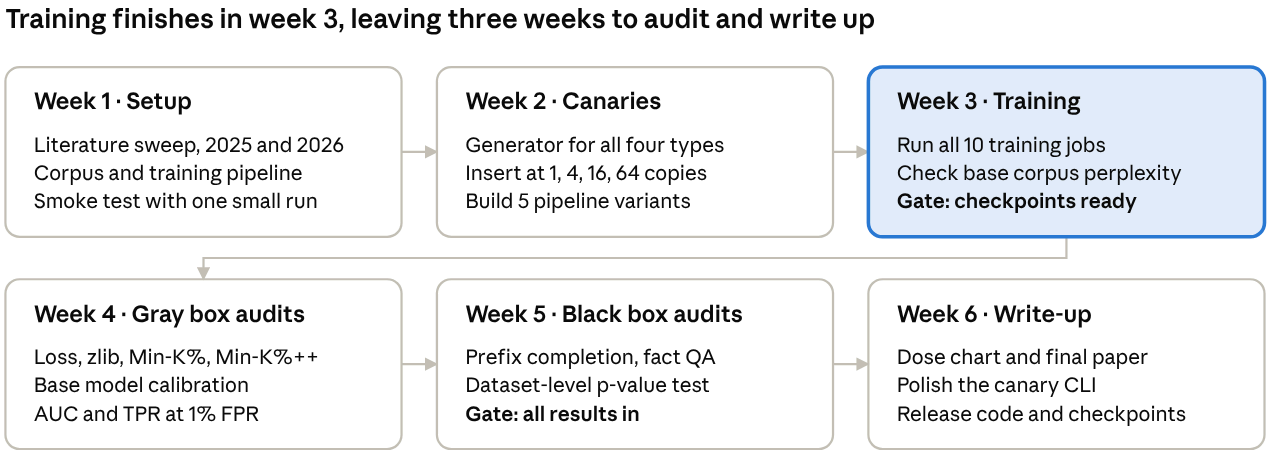
Headline figure: a minimum detectable dose chart, showing the fewest copies needed for $p < 0.01$ per canary type, with one line per pipeline variant.

Deliverables

1. **`canary` Python package and CLI.** `canary generate` makes a creator's canary bundle and its secret held-out set. `canary embed` inserts canaries into documents. `canary audit --model <endpoint>` runs every detector the access level allows and prints a verdict with a p-value.
2. **Audit report.** A one-page scorecard per model: detected or not, p-value, which detectors fired, and confidence.
3. **Final paper.** Results for all four research questions, with the minimum detectable dose chart as the headline figure.
4. **Released artifacts.** The canary generator, trained checkpoints, and evaluation scripts so others can reproduce the results.

Timeline

The plan assumes six weeks until the final deadline.



six week plan · 2 gates

If there is more time, extend week 3 to add the Pythia-1B runs.

Risks and compute

Compute. By a rough estimate (6 × parameters × tokens), one Pythia-410M run on 100M tokens takes about 1 GPU hour on an A100, and a 160M run much less. The full 10-run grid fits in roughly 10 to 15 GPU hours, which a university cluster or Colab Pro can handle.

Risk	Mitigation
Small models memorize too little to detect anything	Keep the 64-copy level as an upper anchor; add more epochs over the canary passages if needed
A 2025 or 2026 paper already covers this	Week 1 literature sweep; if covered, narrow to the fictional fact canary and text-only setting
Fictional facts conflict with real knowledge	Generate fictional entities only, and check they don't appear in the base corpus
Results don't transfer to large commercial models	State this as a limitation; frame findings as lower bounds on detectability
Black box detectors are too weak	Report the negative result honestly; it is itself useful evidence for creators and policymakers

Ethics. All experiments use models we train ourselves. The tool is for creators auditing their own work, and the dataset-level test is designed to keep false accusations rare.

References

Cited from memory; verify titles, venues and years before submitting.

- Carlini, Liu, Erlingsson, Kos, Song. *The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks.* USENIX Security 2019.
- Carlini et al. *Extracting Training Data from Large Language Models.* USENIX Security 2021.
- Kandpal, Wallace, Raffel. *Deduplicating Training Data Mitigates Privacy Risks in Language Models.* ICML 2022.
- Shi et al. *Detecting Pretraining Data from Large Language Models* (Min-K% Prob). ICLR 2024.

- Zhang et al. *Min-K%++: Improved Baseline for Detecting Pre-Training Data from LLMs.* 2024.
- Meeus, Shilov, Faysse, de Montjoye. *Copyright Traps for Large Language Models.* ICML 2024.
- Wei, Wang, Jia. *Proving Membership in LLM Pretraining Data via Data Watermarks.* Findings of ACL 2024.
- Duan et al. *Do Membership Inference Attacks Work on Large Language Models?* COLM 2024.
- Maini et al. *LLM Dataset Inference: Did You Train on My Dataset?* NeurIPS 2024.
- Biderman et al. *Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling.* ICML 2023.